Student Attendance Management System

```
#include <iostream.h>
#include <iomanip.h>
#include <string.h>
#include <conio.h>
#include <stdlib.h>

#define MAX_STUDENTS 30
#define MAX_SUBJECTS 10
#define MIN_ATTENDANCE 75.0


/*============================================================
                    STUDENT CLASS
============================================================*/

class Student
{
public:

   int rollNo;
   char name[50];
   char course[30];

   Student()
   {
      rollNo = 0;
      strcpy(name, "");
      strcpy(course, "");
   }

   void setData(int r, char n[], char c[])
   {
      rollNo = r;
      strcpy(name, n);
      strcpy(course, c);
   }
};
```

```
/*============================================================
                    SUBJECT CLASS
============================================================*/

class Subject
{
public:

   int id;
   char name[40];
   char type[20];

   Subject()
   {
      id = 0;
      strcpy(name, "");
      strcpy(type, "");
   }

   void setData(int i, char n[], char t[])
   {
      id = i;
      strcpy(name, n);
      strcpy(type, t);
   }
};


/*============================================================
                ATTENDANCE LINKED LIST
============================================================*/

class AttendanceList
{
private:

   struct Node
   {
      int subjectId;
      int totalClasses;
      int presentClasses;
      Node *next;
   };
```

```
    Node *head;

public:

    AttendanceList()
    {
        head = NULL;
    }


    void addAttendance(int sid, int total, int present)
    {
        Node *newNode;
        Node *temp;

        newNode = new Node;

        newNode->subjectId = sid;
        newNode->totalClasses = total;
        newNode->presentClasses = present;
        newNode->next = NULL;

        if(head == NULL)
        {
            head = newNode;
        }
        else
        {
            temp = head;


                while(temp->next != NULL)
                {
                        temp = temp->next;
                }

                temp->next = newNode;
            }
    }


    Node *findSubject(int sid)
    {
            Node *temp;
```

```
    temp = head;

    while(temp != NULL)
    {
       if(temp->subjectId == sid)
       {
            return temp;
       }

       temp = temp->next;
    }

    return NULL;
}


int getTotal(int sid)
{
    Node *temp;

    temp = findSubject(sid);

    if(temp == NULL)
    {
       return 0;
    }

    return temp->totalClasses;
}


int getPresent(int sid)
{
    Node *temp;

    temp = findSubject(sid);

    if(temp == NULL)
    {
       return 0;
    }

    return temp->presentClasses;
```

```
}


int getAbsent(int sid)
{
	Node *temp;

	temp = findSubject(sid);

	if(temp == NULL)
	{
	   return 0;
	}

	return temp->totalClasses - temp->presentClasses;
}


float getPercentage(int sid)
{
	Node *temp;

	temp = findSubject(sid);

	if(temp == NULL)
	{
	   return 0;
	}

	if(temp->totalClasses == 0)
	{
	   return 0;
	}

	return ((float)temp->presentClasses /
		temp->totalClasses) * 100;
}


void updateAttendance(int sid, int total, int present)
{
	Node *temp;

	temp = findSubject(sid);
```

```
        if(temp != NULL)
        {
            temp->totalClasses = total;
            temp->presentClasses = present;
        }
    }


    int getOverallTotal()
    {
        Node *temp;
        int total;

        total = 0;
        temp = head;

        while(temp != NULL)
        {
            total = total + temp->totalClasses;
            temp = temp->next;
        }

        return total;
    }


    int getOverallPresent()
    {
        Node *temp;
        int present;

        present = 0;
        temp = head;

        while(temp != NULL)
        {
            present = present + temp->presentClasses;
            temp = temp->next;
        }

        return present;
    }
};
```

```
/*=========================================================
                        ATTENDANCE QUEUE
=========================================================*/

class AttendanceQueue
{
private:

   struct QueueNode
   {
         int rollNo;
         int subjectId;
         char status;
         QueueNode *next;
   };

   QueueNode *front;
   QueueNode *rear;

public:

   AttendanceQueue()
   {
         front = NULL;
         rear = NULL;
   }


   void enqueue(int roll, int sid, char st)
   {
         QueueNode *newNode;

         newNode = new QueueNode;

         newNode->rollNo = roll;
         newNode->subjectId = sid;
         newNode->status = st;
         newNode->next = NULL;

         if(rear == NULL)
         {
            front = newNode;
```

```
            rear = newNode;
        }
        else
        {
            rear->next = newNode;
            rear = newNode;
        }
    }


    void clear()
    {
        QueueNode *temp;

        while(front != NULL)
        {
            temp = front;
            front = front->next;
            delete temp;
        }

        rear = NULL;
    }
};


/*=========================================================
                    ATTENDANCE APPLICATION
=========================================================*/

class AttendanceApp
{
private:

    Student students[MAX_STUDENTS];

    Subject subjects[MAX_SUBJECTS];

    AttendanceList attendance[MAX_STUDENTS];

    AttendanceQueue attendanceQueue;

    int studentCount;
    int subjectCount;
```

```
    int currentStudent;


public:

    AttendanceApp()
    {
        studentCount = 30;
        subjectCount = 10;
        currentStudent = -1;

        initializeStudents();
        initializeSubjects();
        initializeAttendance();
    }


    /*=====================================================
                        INITIALIZE STUDENTS
    =====================================================*/

    void initializeStudents()
    {
        char names[30][50] =
        {
            "Aarav Sharma",
            "Vivaan Patel",
            "Aditya Verma",
            "Arjun Mehta",
            "Rohan Shah",
            "Rahul Joshi",
            "Karan Patel",
            "Akash Sharma",
            "Yash Mehta",
            "Rohit Verma",
            "Amit Joshi",
            "Ankit Shah",
            "Nikhil Patel",
            "Saurabh Singh",
            "Harsh Mehta",
            "Dhruv Sharma",
            "Raj Malhotra",
            "Manav Shah",
```

```
        "Dev Patel",
        "Varun Joshi",
        "Siddharth Verma",
        "Abhishek Singh",
        "Pranav Mehta",
        "Ayush Sharma",
        "Kartik Patel",
        "Sameer Khan",
        "Mohit Joshi",
        "Tushar Verma",
        "Vishal Shah",
        "Om Patel"
    };

    int i;

    for(i = 0; i < 30; i++)
    {
        students[i].setData(
            i + 1,
            names[i],
            "BCA"
        );
    }
}


/*=====================================================
                INITIALIZE SUBJECTS
=====================================================*/

void initializeSubjects()
{
    subjects[0].setData(
        1,
        "DS",
        "Theory"
    );

    subjects[1].setData(
        2,
        "MPMC",
        "Theory"
    );
```

```
subjects[2].setData(
    3,
    "DM",
    "Theory"
);

subjects[3].setData(
    4,
    "SE",
    "Theory"
);

subjects[4].setData(
    5,
    "OE",
    "Theory"
);

subjects[5].setData(
    6,
    "Hindi",
    "Theory"
);

subjects[6].setData(
    7,
    "MP",
    "Practical"
);

subjects[7].setData(
    8,
    "DS Practical",
    "Practical"
);

subjects[8].setData(
    9,
    "MPMC Practical",
    "Practical"
);

subjects[9].setData(
```

```
            10,
            "DM Practical",
            "Practical"
        );
}


/*=====================================================
                INITIALIZE ATTENDANCE
=====================================================*/

void initializeAttendance()
{
        int i;

        for(i = 0; i < studentCount; i++)
        {
            /*
              THEORY - 20 CLASSES
            */

            attendance[i].addAttendance(
                    1,
                    20,
                    10 + ((i * 3) % 11)
            );

            attendance[i].addAttendance(
                    2,
                    20,
                    11 + ((i * 4) % 10)
            );

            attendance[i].addAttendance(
                    3,
                    20,
                    9 + ((i * 5) % 12)
            );

            attendance[i].addAttendance(
                    4,
                    20,
                    12 + ((i * 2) % 9)
            );
```

```
        attendance[i].addAttendance(
                5,
                20,
                10 + ((i * 6) % 11)
        );

        attendance[i].addAttendance(
                6,
                20,
                13 + ((i * 3) % 8)
        );


        /*
          PRACTICAL - 10 CLASSES
        */

        attendance[i].addAttendance(
                7,
                10,
                5 + ((i * 2) % 6)
        );

        attendance[i].addAttendance(
                8,
                10,
                4 + ((i * 3) % 7)
        );

        attendance[i].addAttendance(
                9,
                10,
                5 + ((i * 4) % 6)
        );

        attendance[i].addAttendance(
                10,
                10,
                4 + ((i * 5) % 7)
        );
    }
```

```
/*
  ROLL 6 - RAHUL JOSHI
*/

attendance[5].updateAttendance(1,20,14);
attendance[5].updateAttendance(2,20,15);
attendance[5].updateAttendance(3,20,13);
attendance[5].updateAttendance(4,20,17);
attendance[5].updateAttendance(5,20,16);
attendance[5].updateAttendance(6,20,18);

attendance[5].updateAttendance(7,10,8);
attendance[5].updateAttendance(8,10,7);
attendance[5].updateAttendance(9,10,8);
attendance[5].updateAttendance(10,10,7);


/*
  ROLL 7 - KARAN PATEL
*/

attendance[6].updateAttendance(1,20,12);
attendance[6].updateAttendance(2,20,13);
attendance[6].updateAttendance(3,20,11);
attendance[6].updateAttendance(4,20,15);
attendance[6].updateAttendance(5,20,14);
attendance[6].updateAttendance(6,20,17);

attendance[6].updateAttendance(7,10,7);
attendance[6].updateAttendance(8,10,6);
attendance[6].updateAttendance(9,10,7);
attendance[6].updateAttendance(10,10,6);


/*
  ROLL 8 - AKASH SHARMA
*/

attendance[7].updateAttendance(1,20,10);
attendance[7].updateAttendance(2,20,12);
attendance[7].updateAttendance(3,20,9);
attendance[7].updateAttendance(4,20,13);
attendance[7].updateAttendance(5,20,11);
attendance[7].updateAttendance(6,20,15);
```

```
	attendance[7].updateAttendance(7,10,6);
	attendance[7].updateAttendance(8,10,5);
	attendance[7].updateAttendance(9,10,6);
	attendance[7].updateAttendance(10,10,5);


	/*
	  ROLL 19 - DEV PATEL
	*/

	attendance[18].updateAttendance(1,20,18);
	attendance[18].updateAttendance(2,20,17);
	attendance[18].updateAttendance(3,20,19);
	attendance[18].updateAttendance(4,20,18);
	attendance[18].updateAttendance(5,20,17);
	attendance[18].updateAttendance(6,20,19);

	attendance[18].updateAttendance(7,10,9);
	attendance[18].updateAttendance(8,10,8);
	attendance[18].updateAttendance(9,10,9);
	attendance[18].updateAttendance(10,10,9);
}


/*=====================================================
                    FIND STUDENT
=====================================================*/

int findStudent(int roll)
{
	int i;

	for(i = 0; i < studentCount; i++)
	{
	   if(students[i].rollNo == roll)
	   {
		return i;
	   }
	}

	return -1;
}
```

```
/*=======================================================
                    FIND SUBJECT
=======================================================*/

int findSubject(int sid)
{
      int i;

      for(i = 0; i < subjectCount; i++)
      {
         if(subjects[i].id == sid)
         {
               return i;
         }
      }

      return -1;
}


/*=======================================================
                    SUBJECT LIST
=======================================================*/

void myAttendance()
{
      int choice;

      while(1)
      {
         clrscr();

         cout << "=======================================" << endl;
         cout << "            SUBJECT LIST" << endl;
         cout << "=======================================" << endl;

         cout << endl;

         cout << "THEORY SUBJECTS" << endl;
         cout << "---------------" << endl;

         cout << "1. DS" << endl;
         cout << "2. MPMC" << endl;
```

```
            cout << "3. DM" << endl;
            cout << "4. SE" << endl;
            cout << "5. OE" << endl;
            cout << "6. Hindi" << endl;

            cout << endl;

            cout << "PRACTICAL SUBJECTS" << endl;
            cout << "------------------" << endl;

            cout << "7. MP" << endl;
            cout << "8. DS Practical" << endl;
        cout << "9. MPMC Practical" << endl;
        cout << "10. DM Practical" << endl;

        cout << endl;

        cout << "0. Back" << endl;

        cout << endl;

        cout << "Enter Subject ID: ";
        cin >> choice;

        if(choice == 0)
        {
            return;
        }

        if(choice < 1 || choice > 10)
        {
            cout << endl;
            cout << "Invalid Subject ID!" << endl;
            cout << "Press any key...";
            getch();
        }
        else
        {
            showAttendance(choice);
        }
    }
}
```

```
/*=====================================================
            SHOW ONE SUBJECT ATTENDANCE
=====================================================*/

void showAttendance(int sid)
{
   int subjectIndex;

   int total;
   int present;
   int absent;

   float percentage;

   subjectIndex = findSubject(sid);

   if(subjectIndex == -1)
   {
      cout << endl;
      cout << "Subject not found!";
      getch();
      return;
   }

   total =
      attendance[currentStudent].getTotal(sid);

   present =
      attendance[currentStudent].getPresent(sid);

   absent =
      attendance[currentStudent].getAbsent(sid);

   percentage =
      attendance[currentStudent].getPercentage(sid);

   clrscr();

   cout << "=======================================" << endl;
   cout << "           MY ATTENDANCE" << endl;
   cout << "=======================================" << endl;

   cout << endl;
```

```
cout << "Student Name : "
    << students[currentStudent].name
    << endl;

cout << "Roll Number  : "
    << students[currentStudent].rollNo
    << endl;

cout << endl;

cout << "Subject ID   : "
    << subjects[subjectIndex].id
    << endl;

cout << "Subject      : "
    << subjects[subjectIndex].name
    << endl;

cout << "Type         : "
    << subjects[subjectIndex].type
    << endl;

cout << endl;

cout << "Total Classes   : "
    << total
    << endl;

cout << "Present Classes : "
    << present
    << endl;

cout << "Absent Classes  : "
    << absent
    << endl;

cout << "Attendance      : "
    << setprecision(2)
    << percentage
    << "%"
    << endl;

cout << endl;
```

```
    if(percentage >= MIN_ATTENDANCE)
    {
        cout << "Status          : GOOD"
             << endl;
    }
    else
    {
        cout << "Status          : LOW ATTENDANCE"
             << endl;
    }

    cout << endl;

    cout << "----------------------------------------" << endl;
    cout << "0. Back to Subject List" << endl;
    cout << "----------------------------------------" << endl;

    cout << endl;

    cout << "Press any key to continue...";
    getch();
}


/*================================================
              THEORY ATTENDANCE
================================================*/

void theoryAttendance()
{
    int i;
    int total;
    int present;
    int absent;
    float percentage;

    clrscr();

    cout << "========================================" << endl;
    cout << "          THEORY ATTENDANCE" << endl;
    cout << "========================================" << endl;

    cout << endl;
```

```
for(i = 0; i < subjectCount; i++)
{
    if(strcmp(subjects[i].type, "Theory") == 0)
    {
        total =
            attendance[currentStudent].
            getTotal(subjects[i].id);

        present =
            attendance[currentStudent].
            getPresent(subjects[i].id);

        absent =
            attendance[currentStudent].
            getAbsent(subjects[i].id);

        percentage =
            attendance[currentStudent].
            getPercentage(subjects[i].id);

        cout << subjects[i].id
             << ". "
             << subjects[i].name
             << endl;

        cout << "Total   : "
             << total
             << endl;

        cout << "Present : "
             << present
             << endl;

        cout << "Absent  : "
             << absent
             << endl;

        cout << "Attend. : "
             << setprecision(2)
             << percentage
             << "%"
             << endl;

        cout << "---------------------------------------"
```

```
                << endl;
        }
    }

    cout << endl;
    cout << "Press any key to go back...";
    getch();
}


/*=====================================================
            PRACTICAL ATTENDANCE
=====================================================*/

void practicalAttendance()
{
    int i;
    int total;
    int present;
    int absent;
    float percentage;

    clrscr();

    cout << "=======================================" << endl;
    cout << "         PRACTICAL ATTENDANCE" << endl;
    cout << "=======================================" << endl;

    cout << endl;

    for(i = 0; i < subjectCount; i++)
    {
        if(strcmp(subjects[i].type, "Practical") == 0)
        {
            total =
                attendance[currentStudent].
                getTotal(subjects[i].id);

            present =
                attendance[currentStudent].
                getPresent(subjects[i].id);

            absent =
                attendance[currentStudent].
```

```
            getAbsent(subjects[i].id);

        percentage =
            attendance[currentStudent].
            getPercentage(subjects[i].id);

        cout << subjects[i].id
             << ". "
             << subjects[i].name
             << endl;

        cout << "Total   : "
             << total
             << endl;

        cout << "Present : "
             << present
             << endl;

        cout << "Absent  : "
             << absent
             << endl;

        cout << "Attend. : "
             << setprecision(2)
             << percentage
             << "%"
             << endl;

        cout << "---------------------------------------"
             << endl;
      }
   }

   cout << endl;
   cout << "Press any key to go back...";
   getch();
}


/*=====================================================
            OVERALL ATTENDANCE
=====================================================*/
```

```
void overallAttendance()
{
    int total;
    int present;
    int absent;

    float percentage;

    total =
        attendance[currentStudent].
        getOverallTotal();

    present =
        attendance[currentStudent].
        getOverallPresent();

    absent = total - present;

    if(total > 0)
    {
        percentage =
            ((float)present / total) * 100;
    }
    else
    {
        percentage = 0;
    }

    clrscr();

    cout << "========================================" << endl;
    cout << "          OVERALL ATTENDANCE" << endl;
    cout << "========================================" << endl;

    cout << endl;

    cout << "Student Name : "
         << students[currentStudent].name
         << endl;

    cout << "Roll Number  : "
         << students[currentStudent].rollNo
         << endl;
```

```
    cout << endl;

    cout << "Total Classes   : "
         << total
         << endl;

    cout << "Present Classes : "
         << present
         << endl;

    cout << "Absent Classes  : "
         << absent
         << endl;

    cout << "Attendance      : "
         << setprecision(2)
         << percentage
         << "%"
         << endl;

    cout << endl;

    if(percentage >= MIN_ATTENDANCE)
    {
       cout << "Status          : GOOD"
            << endl;
    }
    else
    {
       cout << "Status          : LOW ATTENDANCE"
            << endl;
    }

    cout << endl;

    cout << "Press any key to go back...";
    getch();
}


/*=====================================================
                MY PROFILE
=====================================================*/
```

```
void myProfile()
{
   clrscr();

   cout << "========================================" << endl;
   cout << "             MY PROFILE" << endl;
   cout << "========================================" << endl;

   cout << endl;

   cout << "Roll Number : "
        << students[currentStudent].rollNo
        << endl;

   cout << "Name        : "
        << students[currentStudent].name
        << endl;

   cout << "Course      : "
        << students[currentStudent].course
        << endl;

   cout << endl;

   cout << "Press any key to go back...";
   getch();
}


/*================================================
            CHANGE ROLL NUMBER
================================================*/

int changeRollNumber()
{
   int roll;
   int index;

   clrscr();

   cout << "========================================" << endl;
   cout << "         CHANGE ROLL NUMBER" << endl;
   cout << "========================================" << endl;
```

```
cout << endl;

cout << "Enter Roll Number" << endl;
cout << "(Enter 0 to go back)" << endl;

cout << endl;

cout << "Roll Number: ";
cin >> roll;

if(roll == 0)
{
   return 0;
}

index = findStudent(roll);

if(index == -1)
{
   cout << endl;
   cout << "Student not found!" << endl;
   cout << "Press any key...";
   getch();

   return 0;
}

currentStudent = index;

cout << endl;

cout << "Roll Number Changed Successfully!"
    << endl;

cout << endl;

cout << "Student: "
    << students[currentStudent].name
    << endl;

cout << endl;

cout << "Press any key...";
getch();
```

```
    return 1;
}


/*=====================================================
                STUDENT PANEL
=====================================================*/

void studentPanel()
{
    int choice;

    while(1)
    {
        clrscr();

        cout << "========================================" << endl;
        cout << "            STUDENT PANEL" << endl;
        cout << "========================================" << endl;

        cout << endl;

        cout << "Welcome, "
             << students[currentStudent].name
             << endl;

        cout << "Roll Number: "
             << students[currentStudent].rollNo
             << endl;

        cout << endl;

        cout << "1. My Attendance" << endl;
        cout << "2. Theory Attendance" << endl;
        cout << "3. Practical Attendance" << endl;
        cout << "4. Overall Attendance" << endl;
        cout << "5. My Profile" << endl;
        cout << "6. Change Roll Number" << endl;
        cout << "0. Back / Logout" << endl;

        cout << endl;

        cout << "Enter your choice: ";
```

```
cin >> choice;

switch(choice)
{
    case 1:

        myAttendance();

        break;


    case 2:

        theoryAttendance();

        break;


    case 3:

        practicalAttendance();

        break;


    case 4:

        overallAttendance();

        break;


    case 5:

        myProfile();

        break;


    case 6:

        changeRollNumber();

        break;
```

```
            case 0:

                return;


            default:

                cout << endl;
                cout << "Invalid choice!";
                getch();
        }
    }
}


/*=====================================================
                STUDENT LOGIN
=====================================================*/

void studentLogin()
{
    char username[30];
    char password[30];

    int roll;
    int index;

    clrscr();

    cout << "=======================================" << endl;
    cout << "            STUDENT LOGIN" << endl;
    cout << "=======================================" << endl;

    cout << endl;

    cout << "Username : ";
    cin >> username;

    cout << "Password : ";
    cin >> password;

    if(strcmp(username, "student") != 0 ||
```

```
   strcmp(password, "1234") != 0)
{
    cout << endl;

    cout << "Invalid Username or Password!"
         << endl;

    cout << endl;

    cout << "Press any key...";
    getch();

    return;
}

cout << endl;

cout << "Login Successful!"
     << endl;

cout << endl;

cout << "Enter Roll Number"
     << endl;

cout << "(Enter 0 to go back)"
     << endl;

cout << endl;

cout << "Roll Number: ";
cin >> roll;

if(roll == 0)
{
    return;
}

index = findStudent(roll);

if(index == -1)
{
    cout << endl;
```

```
        cout << "Student not found!"
             << endl;

        cout << endl;

        cout << "Press any key...";
        getch();

        return;
    }

    currentStudent = index;

    studentPanel();
}


/*=====================================================
                MAIN MENU
=====================================================*/

void run()
{
    int choice;

    while(1)
    {
        clrscr();

        cout << "=======================================" << endl;
        cout << "       STUDENT ATTENDANCE APP" << endl;
        cout << "=======================================" << endl;

        cout << endl;

        cout << "1. Student Login" << endl;
        cout << "2. Exit" << endl;

        cout << endl;

        cout << "Enter your choice: ";
        cin >> choice;

        switch(choice)
```

```
        {
            case 1:

                studentLogin();

                break;


            case 2:

                clrscr();

                cout << endl;
                cout << "Thank you for using" << endl;
                cout << "Student Attendance App!" << endl;

                cout << endl;

                getch();

                return;


            default:

                cout << endl;
                cout << "Invalid choice!";
                getch();
        }
      }
   }
};


/*============================================================
                    MAIN
============================================================*/

void main()
{
   AttendanceApp app;

   app.run();
}
```